

02 继承封装多态与魔法方法

目标：闭卷能写 `User/Admin` 继承题，能读懂封装、多态、多继承、`__add__`、`__del__`。

继承基础【必须闭卷会写】

1. 它解决什么问题

子类复用父类已有属性和方法，再增加自己的内容。实验报告里的 `Admin(User)` 就是典型继承题。

2. 最小语法

```
class User:
    def __init__(self, name):
        self.name = name

    def get_name(self):
        return self.name

class Admin(User):
    pass

admin = Admin("root")
print(admin.get_name())
```

3. 对象创建和调用过程

`Admin` 是子类
`User` 是父类
`Admin` 对象可以使用 `User` 的非私有成员

4. 常见错误

以为子类必须重新写父类所有方法。实际上没有重写时，会去父类找。

5. 考试可能怎么出

定义用户类和管理员类，管理员继承用户并新增权限属性。

6. 最小闭卷题

写 `User` 和 `Admin(User)`，`Admin` 新增 `authority`。

子类初始化与 `super()` 【必须闭卷会写】

1. 它解决什么问题

子类创建对象时，既要初始化父类属性，也要初始化子类自己的属性。

2. 最小语法

```
class User:
    def __init__(self, name, password):
        self.name = name
        self.__password = password

    def get_password(self):
        return self.__password

class Admin(User):
    def __init__(self, name, password, authority):
        super().__init__(name, password)
        self.authority = authority

admin = Admin("root", "123456", "系统管理")
print(admin.name)
print(admin.get_password())
print(admin.authority)
```

3. 对象创建和调用过程

```
Admin(...)
→ Admin.__init__
→ super().__init__(name, password)
→ 调用 User.__init__ 保存 name 和 __password
→ self.authority 保存管理员权限
```

4. 常见错误

```
super().__init__(name, password)
```

错误： `super` 少了括号。

```
super().__init__
```

错误：只拿到了方法，没有调用，缺少 `()`。

5. 考试可能怎么出

改错题：子类漏调父类初始化，导致父类属性不存在。

6. 最小闭卷题

`Admin(User)` 中调用 `super().__init__()`，新增 `authority`。

方法重写【必须闭卷会写】

1. 它解决什么问题

子类需要改变父类同名方法的行为。

2. 最小语法

```
class User:
    def show_info(self):
        return "普通用户"

class Admin(User):
    def show_info(self):
        return "管理员"

print(Admin().show_info())
```

3. 对象创建和调用过程

`Admin` 对象调用 `show_info`
→ 先找 `Admin` 自己

- 找到同名方法
- 执行子类版本

4. 保留父类逻辑再扩展

```
class User:
    def __init__(self, name):
        self.name = name

    def show_info(self):
        return f"name={self.name}"

class Admin(User):
    def __init__(self, name, authority):
        super().__init__(name)
        self.authority = authority

    def show_info(self):
        base = super().show_info()
        return f"{base}, authority={self.authority}"
```

5. 考试可能怎么出

阅读题：父类和子类都有同名方法，判断输出。

6. 最小闭卷题

`Animal.cry()` 返回 "...", `Dog.cry()` 返回 "汪汪"。

封装【必须闭卷会写】

1. 它解决什么问题

限制外部随便修改内部数据，通过方法或属性接口做校验。

2. 最小语法

```
class Person:
    def __init__(self, age):
        self.__age = age
```

```
def get_age(self):  
    return self.__age  
  
def set_age(self, age):  
    if 0 <= age <= 120:  
        self.__age = age
```

3. 对象创建和调用过程

__age 被名称改写
外部不直接 p.__age
通过 get_age/set_age 访问和修改

4. Python 封装三种写法

写法	含义	考试要求
name	公开属性	会写
_name	受保护约定	会读
__name	名称改写	会写
@property	属性式访问	会读会改

5. @property 最小例子

```
class Person:  
    def __init__(self, age):  
        self.__age = age  
  
    @property  
    def age(self):  
        return self.__age
```

6. 最小闭卷题

写 Course，__location 私有，通过 get_location() 返回。

多态与鸭子类型【必须会读会改】

1. 它解决什么问题

同一个函数接收不同对象，调用同名方法，得到不同行为。

2. 最小语法

```
class Animal:
    def speak(self):
        return "..."
```



```
class Dog(Animal):
    def speak(self):
        return "汪汪"
```



```
class Cat(Animal):
    def speak(self):
        return "喵喵"
```



```
def make_sound(animal):
    print(animal.speak())
```



```
make_sound(Dog())
make_sound(Cat())
```

3. 对象创建和调用过程

```
make_sound(Dog())
```

→ animal 是 Dog 对象
→ animal.speak() 执行 Dog 版本

4. 鸭子类型

Python 更关注“有没有这个方法”，不强制一定继承同一个父类。

5. 考试可能怎么出

阅读题：不同子类重写同名方法，判断输出。

6. 最小闭卷题

定义 Shape、Circle、Rectangle，让它们都有 area()。

多继承与 MRO 【必须会读会改】

1. 它解决什么问题

一个类同时继承多个父类。考试只要求看调用顺序，不深入 C3 算法。

2. 最小语法

```
class A:
    def show(self):
        return "A"

class B:
    def show(self):
        return "B"

class C(A, B):
    pass

c = C()
print(c.show())
print(C.__mro__)
```

3. 对象创建和调用过程

```
C(A, B)
→ 找 show 时先 C
→ 再 A
→ 再 B
→ 再 object
```

4. 常见错误

以为多继承一定按“最近父类名字”找。实际看继承括号里的顺序和 MRO。

5. 考试可能怎么出

程序输出题：class C(A, B) 和 class C(B, A) 输出不同。

6. 最小闭卷题

写 `Flyable.run`、`Swimmable.run`，`Duck(Flyable, Swimmable)` 调用 `run()`。

运算符重载 `__add__` 【必须会读会改】

1. 它解决什么问题

让自定义对象也能使用 `+`。

2. 最小语法

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)
```

3. 对象创建和调用过程

```
p1 + p2
→ p1.__add__(p2)
→ 返回一个新的 Point
```

4. 常见错误

```
def __add__(self, other):
    Point(self.x + other.x, self.y + other.y)
```

少了 `return`，结果是 `None`。

5. 考试可能怎么出

改错题：定义了 `Score`，但 `s1 + s2` 报错。

6. 最小闭卷题

写 `Score.__add__`，让 `Score(80) + Score(10)` 得到 `Score(90)`。

`__eq__` 了解即可，用于控制 `==`。

析构方法 `__del__` 【必须会读会改】

1. 它解决什么问题

对象准备被销毁时，Python 会尝试调用 `__del__`。考试一般考触发时机和输出顺序。

2. 最小语法

```
class Demo:
    def __init__(self, name):
        self.name = name
        print("创建", self.name)

    def __del__(self):
        print("销毁", self.name)

d = Demo("A")
del d
```

3. 对象创建和调用过程

```
Demo("A") → __init__
del d → 删除引用
对象没有引用后 → 尝试执行 __del__
```

4. 必须强调

触发时间不稳定
不能依赖 `__del__` 完成关键业务逻辑
循环引用可能影响触发
文件资源优先使用 `with`

5. 考试可能怎么出

阅读题：创建对象、删除对象，判断输出顺序。

6. 最小闭卷题

写 `FileDemo`，`__init__` 打印“创建”，`__del__` 打印“销毁”。

组合关系【必须会读会改】

1. 它解决什么问题

对象里面保存另一个对象或对象列表。

2. 最小语法

```
class Classroom:
    def __init__(self):
        self.students = []

    def add_student(self, student):
        self.students.append(student)
```

3. 继承 vs 组合

继承是“是一个”：Admin 是 User

组合是“有一个”：Classroom 有 students

抽象类【了解即可】

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

抽象类规定子类必须实现某些方法，未实现时不能实例化。期末只需会读，不安排复杂手写。

综合示例清单

- Circle：见第一份笔记，闭卷会写。
- Course：私有属性 `__location`。
- User/Admin：继承和 `super()`。
- Animal/Dog/Cat：多态。
- Point.__add__：运算符重载。
- 对象列表排序：`key=lambda student: student.score`。

继承关系执行图

```
Admin("root", "123", "系统管理")
→ Admin.__init__
→ super().__init__("root", "123")
→ User.__init__
→ 保存 name 和 __password
→ 回到 Admin.__init__
→ 保存 authority
```

super 调用图

```
子类方法
→ super().父类方法()
→ 父类逻辑
→ 回到子类继续扩展
```

封装方式对照

写法	作用	优先级
<code>self.name</code>	公开属性	必须会写
<code>self._name</code>	受保护约定	会读
<code>self.__name</code>	名称改写	必须会写
<code>@property</code>	属性式访问	会读会改

魔法方法速查

方法	触发
<code>__init__</code>	创建对象
<code>__str__</code>	<code>print(obj)</code>
<code>__repr__</code>	调试显示
<code>__add__</code>	<code>obj1 + obj2</code>
<code>__del__</code>	对象销毁前

12 个易错点

1. 子类漏调 `super().__init__()`。
2. `super.__init__` 少括号。
3. 子类重写后优先执行子类方法。
4. 私有属性不是绝对安全，是名称改写。
5. `@property` 访问不加括号。
6. 多态看对象实际类型。
7. 多继承看 MRO。
8. `__add__` 要返回结果。
9. `__del__` 不能做关键资源清理。
10. 继承是“是一个”，组合是“有一个”。
11. 抽象类不实现抽象方法不能实例化。
12. 对象列表排序要写 `key`。

10 个口头自测

1. 子类为什么要调用 `super().__init__()`？
2. 什么叫方法重写？
3. `Admin(User)` 谁是父类？
4. 私有属性 `__password` 子类能直接访问吗？
5. `@property` 为什么不用括号？
6. 多态的核心是什么？
7. `C(A, B)` 同名方法先找谁？
8. `p1 + p2` 会调用什么？
9. `del obj` 一定立刻执行 `__del__` 吗？
10. 继承和组合怎么区分？

6 个最小闭卷题

1. 写 User/Admin 继承。
2. 写 Animal/Dog/Cat 多态。
3. 写 Person.__age 和 get_age/set_age。
4. 写 Duck(Flyable, Swimmable) 并查看 MRO。
5. 写 Score.__add__。
6. 写 Classroom 保存学生对象列表。

答案区

```
class User:
    def __init__(self, name, password):
        self.name = name
        self.__password = password

    def get_password(self):
        return self.__password

class Admin(User):
    def __init__(self, name, password, authority):
        super().__init__(name, password)
        self.authority = authority

    def show_info(self):
        return f"{self.name}:{self.authority}"
```

[进入 OOP 闭卷训练](#)